

Algorithmic Analysis of SARS-CoV-2 Sequences

Advanced Algorithms Design (TC2038), Situation Problem 1

Santiago Hernández

A01787550, Tecnológico de Monterrey

September 22, 2026

One question, six files

Wuhan 2019 (MN908947.3) vs Texas 2020 (MT106054.1): where are the genomes the same, where do they differ, and do the differences change amino acids?

Every technique of the course gets one concrete question about these files; all are compared on measured cost against a brute-force baseline.

file	length	symbols	most	least	unexpected
SARS-COV-2-MN908947.3.txt	29903	4	T	C	none
SARS-COV-2-MT106054.1.txt	29882	4	T	C	none
gen-ORF1AB.txt	21290	4	T	C	none
gen-S.txt	3822	4	T	G	none
gen-M.txt	669	4	T	G	none
seq-proteins.txt	9701	20	L	W	none

Question to technique

question	technique	why it fits
Where are M, S and ORF1ab?	search: naive, KMP, Z	headers give ground truth
How similar, what changed?	DP (LCS) + Hirschberg	exact; memory is the wall
Most non-overlapping ORFs?	greedy selection	earliest end is optimal
Where is each protein?	backtracking	genome prunes 6^L paths
Top k-mer, one mismatch?	branch and bound	alive count is the bound
Longest palindrome per gene?	Manacher vs centre exp.	mirror reuse, $O(n)$

Flagship: LCS by DP, made feasible by Hirschberg

$$L(i,j) = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ 1 + L(i-1, j-1) & x_i = y_j \\ \max\left(L(i-1,j),\, L(i,j-1)\right) & x_i \neq y_j \end{cases}$$

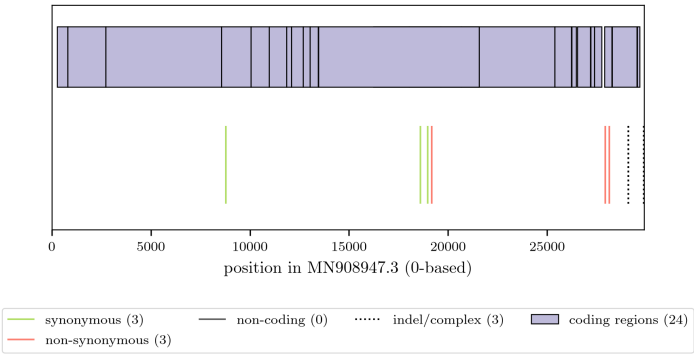
Dynamic programming: the wall

- $\Theta(nm)$ time; the table has $29\,903 \times 29\,882 \approx 8.9 \times 10^8$ cells: **3.6 GB**.
- Two rows give the length in 239 KB, but no traceback.

Hirschberg: divide and conquer

- **divide:** forward last row of X[:mid] plus backward last row of X[mid:] picks the split k of Y
- **conquer, combine:** recurse on both rectangles, concatenate
- $O(n+m)$ **memory**, at most $2nm$ work, depth $O(\log n)$
- on the genomes: 255 splits, depth 7, 3.9 s

Wuhan vs Texas: $|LCS| = 29\,875$, similarity 0.99906



Differences along MN908947.3; coding regions from backtracking.

Wuhan length	29 903
Texas length	29 882
$\|LCS\|$	**29 875**
similarity	**0.99906**
substitutions	6
synonymous	3
non-synonymous	3
indels	3
Hirschberg time	3.9 s
DP table (not run)	3.6 GB

The six substitutions: three silent, three change the protein

Wuhan pos	Texas pos	base	region	codon	amino acid	effect
8782	8782	C→T	QHD43415_4	AGC→AGT	S→S	synonymous
18603	18603	T→C	QHD43415_13	CAT→CAC	H→H	synonymous
18975	18975	T→A	QHD43415_13	GTT→GTA	V→V	synonymous
19175	19175	A→C	QHD43415_13	GAT→GCT	D→A	non-synonymous
27925	27925	C→T	QHD43422	ACA→ATA	T→I	non-synonymous
28144	28144	T→C	QHD43422	TTA→TCA	L→S	non-synonymous

- The three indels sit after 29 000: a 21-nucleotide poly-A run missing from Texas (how far the read went, not a mutation), and an indel pair three positions apart that is really one more substitution, C→T at 29 095. The LCS has no substitution operation, so an indel pair and a substitution can tie: the limit of an LCS as a diff.
- The last two rows fall outside the three gene files; they were classified only after backtracking located QHD43422.

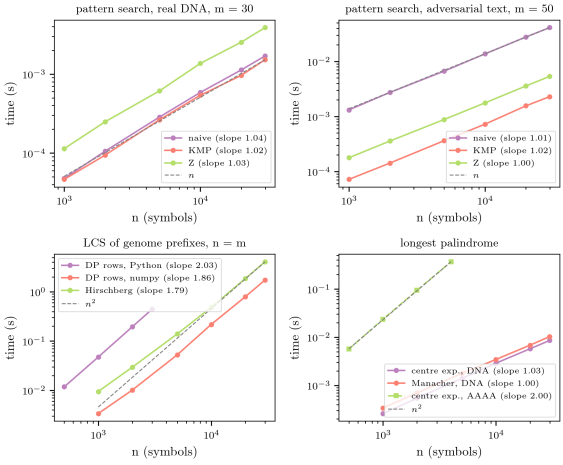
Backtracking found the ribosomal frameshift

protein	start	matched
QHD43415_10	13024	139/139
QHD43415_11	13441	**9/932**
QHD43415_12	16236	601/601

Three of the 24 records, 0-based positions.

- 23 of 24 proteins located over their full length, a few dozen nodes each.
- The exception matches exactly 9 amino acids from 13 441, that is up to nucleotide 13 468: the ORF1a / ORF1b ribosomal frameshift, where the reading frame moves by one base.
- Translating ORF1ab in both frames finds 10 records whole in the first and 4 in the second, which validates the frames and the codon table.
- The search did not fail; it found a fact about the data.

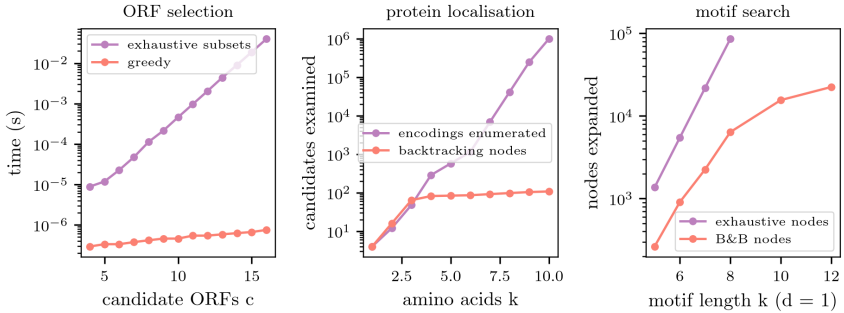
Running time against input size



Log-log; dashed lines are the theoretical growth.

- **Search, real DNA:** all linear; naive is not slower because a mismatch ends most alignments after one or two comparisons.
- **Adversarial A^n:** naive keeps slope 1 but runs 20× slower than KMP, the factor $m = 50$.
- **LCS:** slope ≈ 2 for all three; Hirschberg $\approx 2\times$ numpy rows, the $2nm$ bound; pure Python reaches only $n = 3\,000$ in the time numpy takes for the genomes.
- **Palindromes:** centre expansion linear on DNA, quadratic on A^n; Manacher linear on both.

Search-space techniques against their exhaustive baselines



- **Greedy:** exhaustive subsets double with every candidate ORF; greedy is flat.
- **Backtracking:** encodings grow with codon degeneracy, 10^6 at $k = 10$; nodes stay near 100 because three amino acids pin one genome position.
- **Branch and bound:** same optimum as exhaustive search (TGTTGTTA, 75 hits) with 7% of the nodes at $k = 8$; still runs at $k = 12$, where exhaustive search cannot.

Which technique won, and why

Winner: divide and conquer over DP

- Hirschberg turned an infeasible run into a feasible one: a 3.6 GB table became $O(n + m)$ rows and 3.9 s for two full genomes.
- Memory bound changed, time bound not: still $O(nm)$, about $2\times$ the length-only rows, exactness never traded.
- Every other technique reached an answer brute force could also reach, given time; this one reached an answer brute force could not.

Across techniques

- **Scaled best:** linear string algorithms and greedy, slope ≈ 1 on every input.
- **Cost most:** LCS at slope 2, and every exhaustive baseline: 2^c, 6^k, 4^k.
- **Datasets flipped behaviour:** adversarial input flips naive vs KMP ($20\times$) and centre expansion vs Manacher (slope 1 to 2); the codon threshold flips greedy.
- **Pruning saves nodes, not time:** backtracking because the genome pins the prefix; branch and bound only as good as its bound.

Run the baseline. Slopes reproduce the exponents, constants decide the run, and only the baseline says whether the clever algorithm was needed.